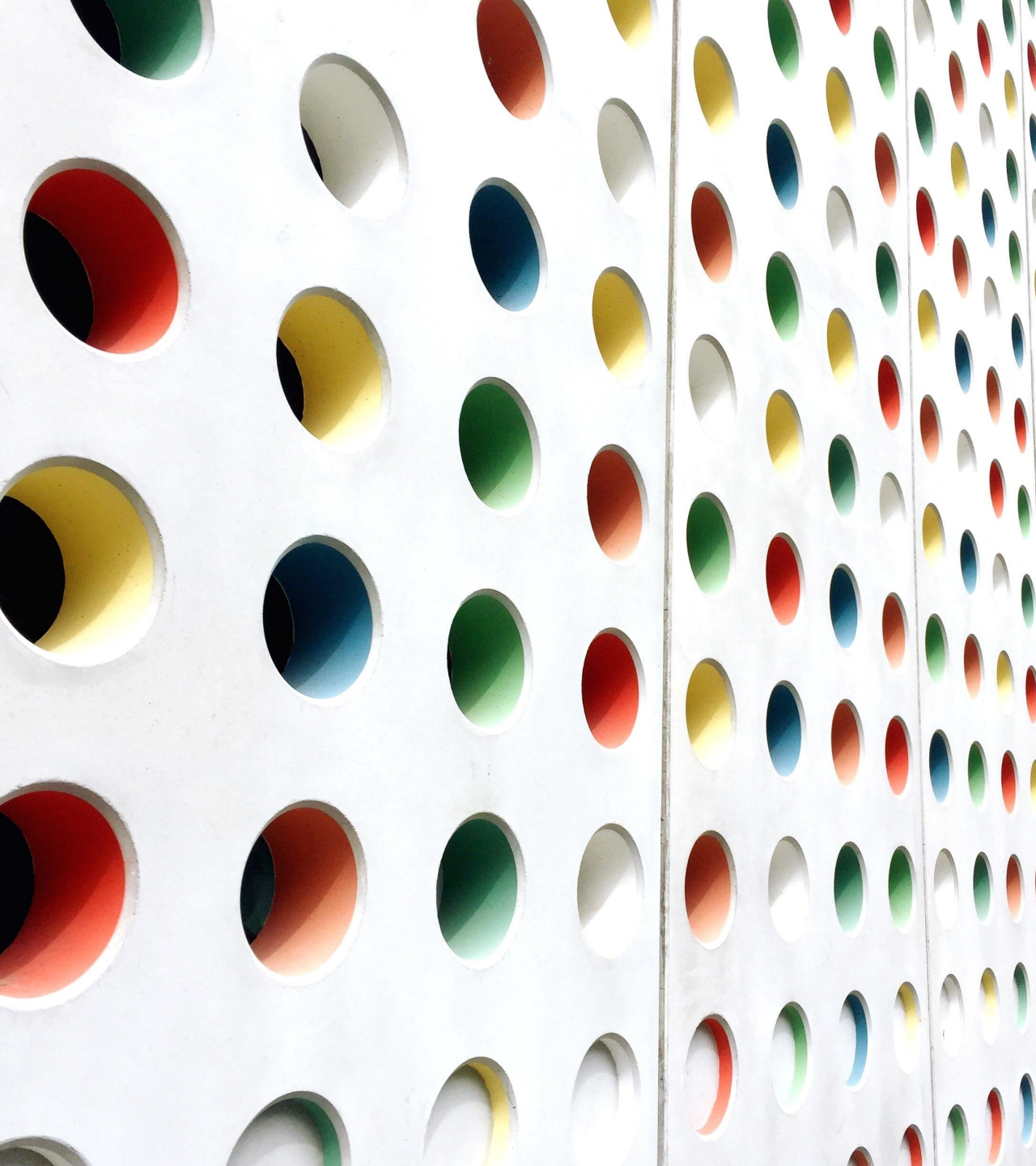




Requirements for Semantic Web Architecture and Building Blocks

BY

SANA RIZWAN

Requirements for Semantic Web Architecture

1. Cover wide range of domain
2. Involving large amount of people and organizations
3. Fast growth

Important for Web success

- Many people could set up a Web server easily and independently from each other.
- More people could create documents, put them online, and link them to each other.
- Even more people could use a browser and access any Web server (and actually some other ones, like FTP or gopher servers) for retrieving these documents.

The fact that each individual system has been coupled only very loosely with the other one and that document creation, document delivery, and document browsing could happen in isolation in each of the many individual nodes was of key importance for enabling the fast adoption of the early Web.

- The network was partially slow (“World Wide Wait”), while other parts might still operate at full speed
- Single Web servers broke, because others would still work
- Hyperlinks were broken, because others would still lead you somewhere



User
Experience

Web's architecture allowed for fast growth and adoption, while other, technically more elaborate systems, such as hypertext systems from the 1980s, lacked the capabilities for mass contribution and gradual degradation. The lesson to be learned is that a state-of-the-art system that produces higher quality output (e.g., no dangling links) may be less useful to its users than a less consistent system, which achieves a separation of concern, letting users do what they are most concerned about, that is, easily create and access documents. Furthermore, a distributed system without the need for a central coordinator is inherently robust. While there are many potential problems that may affect individual nodes in the World Wide Web, the only problem leading to brittleness in the World Wide Web as a whole is the hierarchical control of the IP addresses and the Domain Name System of the Internet.

WWW Problems

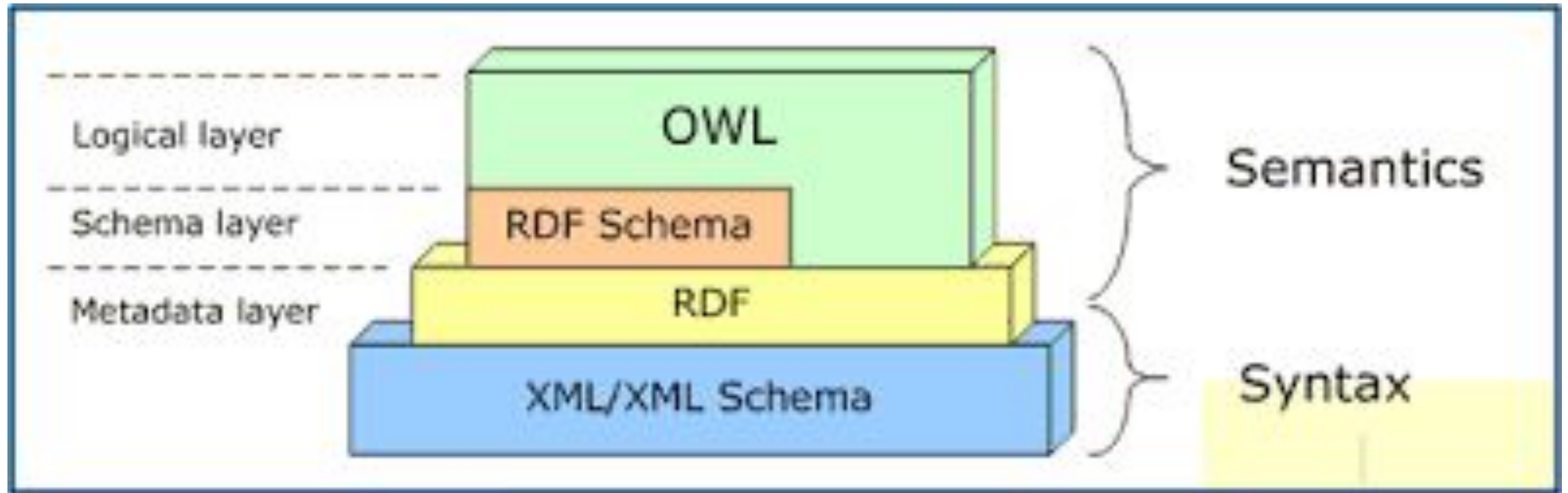
Reasons to define Principles for SW

- Unlike traditional information systems design, both the **Web** and the **Web of Data** do not emphasize the specific application, but rather generic needs that can be realized in many different components which only agree on a core set of standards.
- So that the development of the architecture of the Semantic Web first focused on protocols and languages, like HTTP (Hypertext Transfer Protocol) was mostly accepted as given for the Semantic Web in order to be **compatible** with the Web, but where the language layer was considered pivotal to the **flexibility of creating and exploiting** a Web of Data.

Architectural viewpoints

1. First architectural viewpoint presented is the famous Semantic Web layer cake which is a Semantic Web **protocols** and **languages** layer cake. Most of the parts in the layer cake are related to **data publishing and exchange**.
2. Second architectural viewpoint considered is a functional view; a description of software components that provide certain functionality, such as **supporting language standards** or **user interaction** and **alternative architectural choices** for Semantic Web applications.



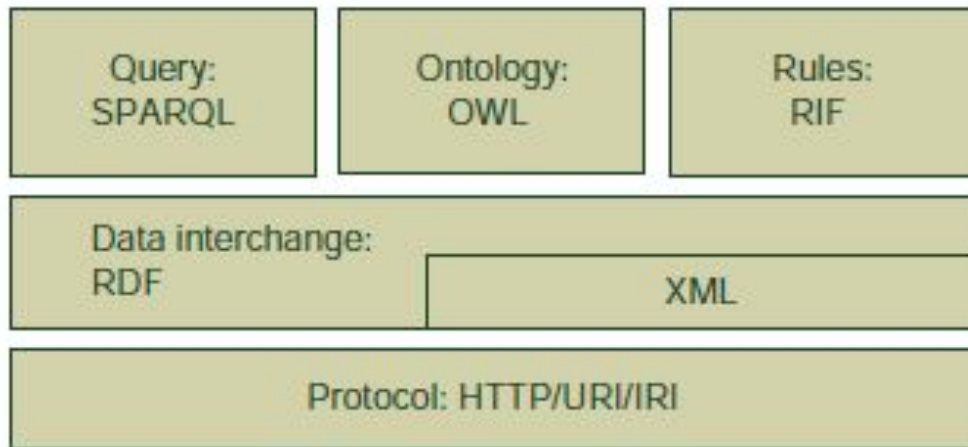


Layer Division SW

Protocol and Languages

- Focus should be on established languages.
- Given the decentralized nature of the Semantic Web, data publishers require a way to unambiguously refer to resources. Resources on the Internet are identified with Uniform Resource Identifiers (URIs).
- URIs on both the Web and the Semantic Web typically use identifiers based on HTTP, which allows for piggybacking on the Domain Name System (DNS) to ensure the global uniqueness of domain names and hence URIs.
- URIs is a mechanism for retrieving content; a user can dereference the URI, a lookup using HTTP. Given there is content hosted at the URI, the user retrieves content which is associated with the URI.
- The ability to unambiguously point to resources and dereference them is a first step.
- Next, required is a language to exchange description of resources. The Extensible Markup Language (XML) is used for encoding documents and provides means for specifying and serializing structured documents which can be parsed across operating systems.
- Building on a referencing and a document exchange mechanism, means to encode descriptions about resources are required. Given that the data on the Semantic Web are highly distributed, the description of resources should be encoded in a way that facilitates integration from many sources. A graph-structured data format achieves the easy integration of data from multiple sources.

Semantic Web Protocol and Language Standards

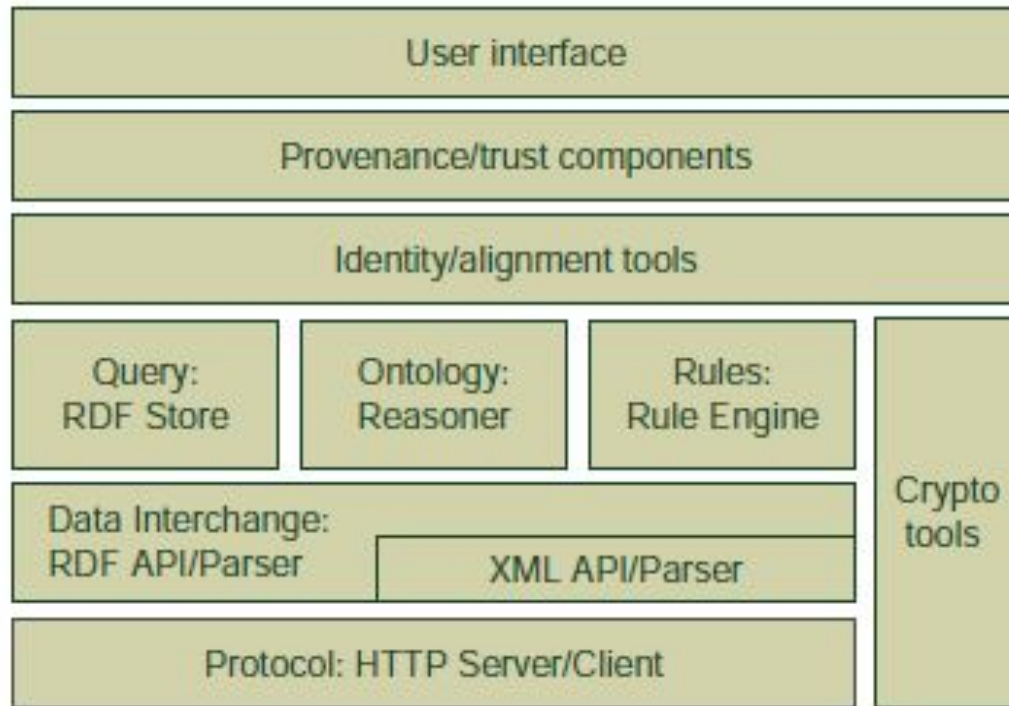


- W3C standard for encoding such data is the Resource Description Framework (RDF). RDF graphs can be serialized in multiple ways; one of the most used is the XML serialization. Having integrated data, mechanisms for querying the integrated graphs are necessary.
- SPARQL (a recursive acronym for SPARQL Protocol and RDF Query Language) is a declarative query language (like SQL) which allows for specifying queries against data in RDF.
- Encoding data as graph covers only parts of the meaning of the data. Often, constructs to model class or property hierarchies provide machines and subsequently humans a more comprehensive understanding of data. To more comprehensively model a domain of interest, so-called ontology languages can be employed.

Semantic Web Protocol and Language Standards Conti...

- RDF Schema (**RDFS**) is a language which can be used to express, for example, class and property hierarchies as well as domain and range of properties. Since data originate from multiple sources and are thus highly heterogeneous, means to reconcile data from different sources and to check consistency of combined data are required.
- The Web Ontology Language (**OWL**) allows for specifying the equality of resources or cardinality constraints of properties, for example. Ontology languages allow for automated inferences, that is, drawing conclusions based on existing facts.
- An alternative way to specifying logical inferences is via rules. Often, users require means to express logical rules which transform data or enrich data with additional specifications. The Rules Interchange Format (**RIF**) allows for encoding and exchanging such logical rules.

Software Components



- **URI** can be used to retrieve content pertaining to the resources they identify. **Web servers** provide the infrastructure for serving content via HTTP, and **HTTP clients** provide lookup functionality.
- Semantic Web inherits the basic infrastructure for referencing and lookup from the Web.
- Content returned as the result of HTTP lookups can be processed using XML parsers and APIs (in case of non-RDF content) or RDF parsers and APIs.
- While W3C developed a specification for manipulating XML documents (the Document Object Model), no such standardized specification exists for processing RDF content.

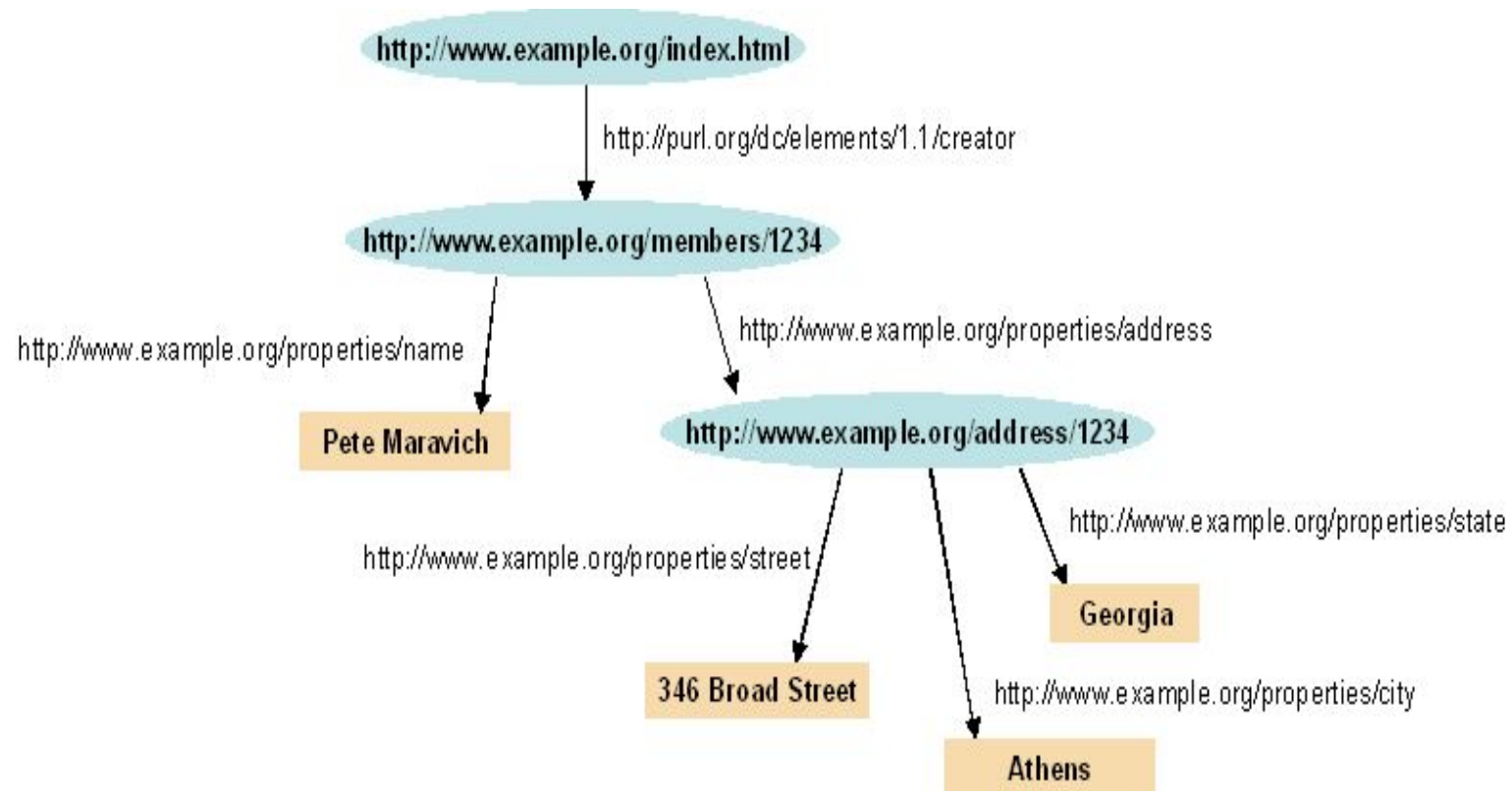
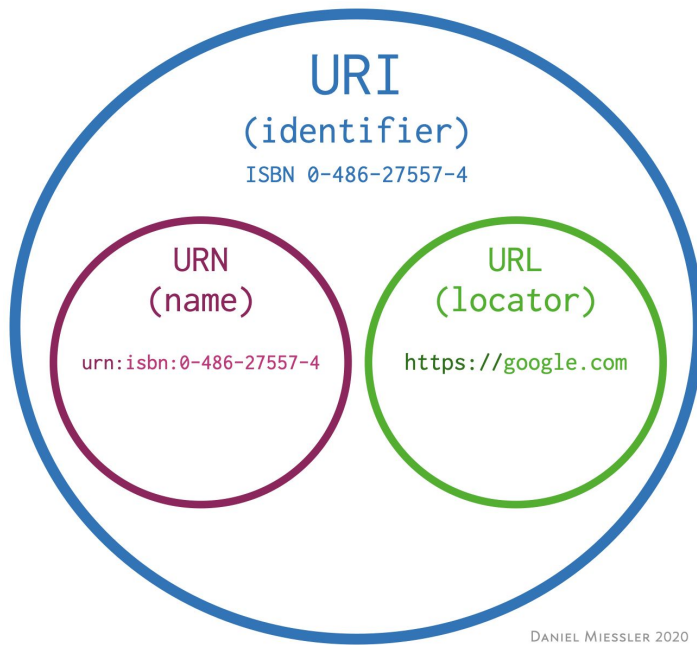
Software Components Conti...

- Having gathered RDF data, typical applications use **RDF repositories** for **storing and indexing RDF data** and **processing SPARQL queries**. Ontology languages, such as **RDFS and OWL**, add more expressivity to RDF data, and so-called **reasoners** are able to process ontologies and **draw** specified inferences. Similarly, rule engines allow for processing **rules exchanged** in **RIF**.
- A crucial point for transmitting sensitive (personal) data is to ensure that data transmissions cannot be **intercepted, read, or altered**. **Crypto-tools** cover **encryption and authentication** technologies to ensure the secure exchange of data. Crypto-modules, such as **SSL processors**, verify **digital certificates** and provide cryptographic **privacy and authentication**.
- Given that content aggregated from many sources often uses multiple identifiers to denote the same real-world object, an **integration and alignment** layer provides for consolidation and the **tighter integration of data**. The **provenance and trust** layer analyzes the data in conjunction with additional information to provide the user with a notion of trust associated to individual data items.
- Finally, the **user interface** enables users to **interact** with Semantic Web data. From a **functionality viewpoint**, some user interfaces are generic and operate on the **graph structure** of the data, while others are tailored to a certain **domain and ontology**.

Building Blocks

1. Referencing, Transport Protocol, and Linked Data Principles

- Adopted traditional client–server model.
- Documents on servers are interlinked with each other in a decentralized manner, and clients use these links to navigate from server to server.
- Model has been condensed to the necessary functionality, and loose coupling between one server and another is achieved by (unilaterally) linking from a document on one server to a document on another server.
- The referenced document is located on the same server or a server on a different continent does not make a fundamental difference in the process of linking.
- URI/IRI and HTTP are the core specifications for both the Web and the Semantic Web and enable decentralized referencing and transport of content.
- URI starts with a scheme name (e.g., http, ftp, tel, mailto) followed by additional information.



URI

Data Publishers

Data publishers on the Semantic Web typically use linked data principles

1. “Use URIs as names for things.
2. Use HTTP URIs so that people can look up those names.
3. When someone looks up a URI, provide useful information, using the standards (RDF, SPARQL).
4. Include links to other URIs, so that they can discover more things.”

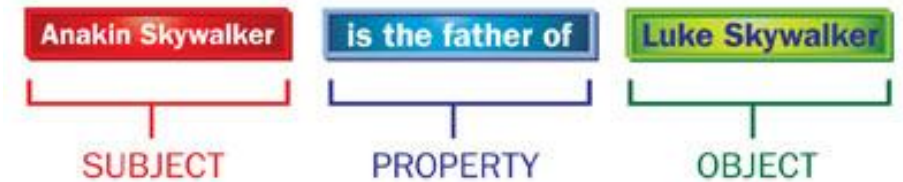
Linked data cloud of Semantic Web data

- Data publishing adhering to these simple principles leads to a unified system for data access and interlinkage. All data publishers use these standards, data consumers can use those standards (as clients) to access the data and further process the data. Like Wrapper
- D2R provides a mechanism for exposing data stored in relational databases as linked data. Custom-built wrapper software often provides linked data access to Web sources returning JSON or XML.
- HTTP is based on the representational state transfer (REST) request/response model, which can be extended to query/answer for data. This model decouples data providers from data consumers, allowing clients to use uniform interfaces for accessing and querying data stored on remote servers.
- In the RESTful approach, information about resources identified by URIs can be accessed directly using simple HTTP GET method, or if a site implements a SPARQL endpoint, clients can pose powerful SPARQL queries against the published data.

2. Data Interchange

- Graph-structured data models have been identified as candidates for integration of data from disparate sources.
- RDF is such a graph structured data model that uses URIs as identifying mechanism and provides the means for the separate modeling and publishing of data which can be interlinked and integrated in subsequent steps.
- RDF is a data format based on directed labeled graphs, which contains data items consisting of (subject, predicate, object) triples.
- `RDF:type` <-- RDF property

How Semantic Web Works



How Semantic Web Works



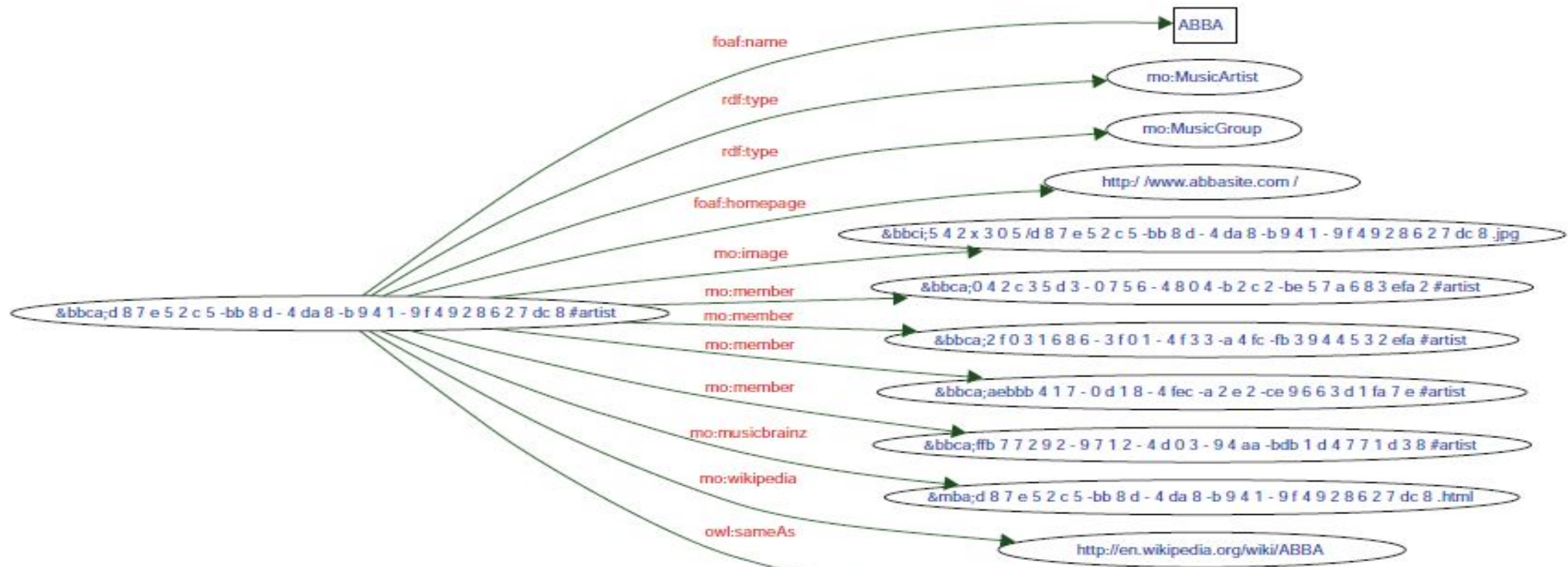
Data Interchange -- Serialization

- RDF is an abstract data format which can be written in multiple serializations. One of the serializations is XML, the Extensible Markup Language.
- Other serializations include Turtle, a format used in which encodes RDF in a triple language and allows for shortcuts, such as “;” for repeating subjects or “,” for repeating subject/predicate pairs.
- A recent development is RDFa which allows for embedding RDF statements directly into (X)HTML documents.
- To be able to process RDF in application programs, parsers such as **ARP** from the Jena project or parsers part of the **Redland RDF library** can be used. These libraries also contain repositories which allow for storing and retrieving RDF.

3. Querying, Updating, and Views

- The SPARQL Query Language for RDF is designed for evaluating queries against RDF datasets and designed to handle complex structure queries, typically over data stored in RDF repositories.
- The repository that supports SPARQL must implement the querying of the underlying data using a specific syntax and protocol.
- With RDF repositories, access to the information is no longer realized by dereferencing individual URIs for RDF files, but by posing queries to SPARQL endpoints.
- SPARQL allows a user to specify arbitrary URIs (even those not accessible on the Web) and a graph pattern that should be matched against the knowledge base together with additional constraints.

RDF Graph -- URIs are abbreviated using RDF/XML syntax

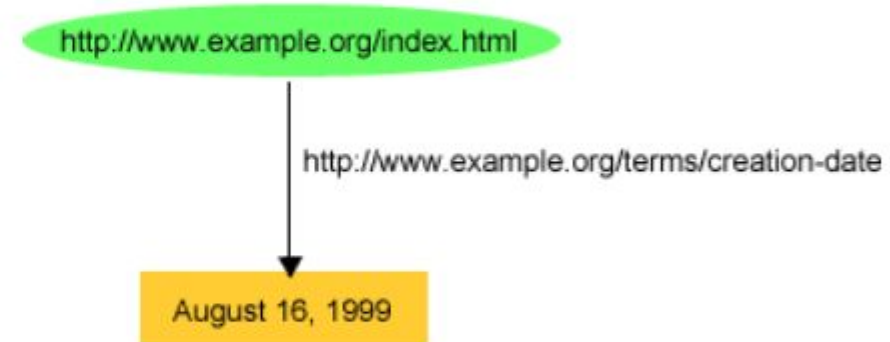


Turtle-- Terse RDF Triple Language

- Concrete syntax for RDF.
- Turtle provides textual syntax for writing down and exchanging RDF graphs. Syntax includes the triple notation used in the document so far, but also includes some additional syntactic means to simplify the specification of RDF graphs.

`http://www.example.org/index.html` has a `creation-date` whose value is `August 16, 1999`

The RDF graph for this single statement, after assigning a URIref to the `creation-date` property



Describing a Web Page's Creation Date

with a triple representation of:

```
ex:index.html    ex:terms:creation-date    "August 16, 1999" .
```

Turtle for the web page

Turtle for the Web Page's Creation Date

```
@prefix rdf:    <http://www.w3.org/1999/02/22-rdf-syntax-ns#>.
@prefix exterms: <http://www.example.org/terms/>.

<http://www.example.org/index.html> exterms:creation-date "August 16, 1999".
```

The only additional syntactic element to what has been used so far is the usage of the `@prefix` keyword in first two lines. These are the namespace declarations, ie, the association of a prefix used in the document with a URI; one line per prefix.

The Turtle syntax provides a number of abbreviations to make common uses easier to write. For example, it is typical for the same resource to be described with several properties and values at the same time. To handle such cases, Turtle allows multiple property elements representing those properties to be nested within the symbol element that identifies the subject resource. For example, to represent the following group of statements about `http://www.example.org/index.html`:

```
ex:index.html  dc:creator      exstaff:85740 .
ex:index.html  exterms:creation-date "August 16, 1999" .
ex:index.html  dc:language      "en" .
```

FOAF— Friend of a Friend

- The FOAF ("Friend of a Friend") project is a community driven effort to define an RDF vocabulary for expressing metadata about people, and their interests, relationships and activities.
- FOAF is an open community-lead initiative which is tackling head-on the wider Semantic Web goal of creating a machine processable web of data.
- Being an RDF application means that FOAF can claim the usual benefits of being easily harvested and aggregated. And like all RDF vocabularies it can be easily combined with other vocabularies, allowing the capture of a very rich set of metadata.
- Definitions of all classes and properties defined in the associated RDF schema.
- Focus on two of the most commonly used classes it defines: **Person** and **Image**. The remaining definitions cover the description of documents, projects, groups, and organizations; consult the specification for more information.

FOAF Metadata Example

Personal Metadata

The Person class is the core of the FOAF vocabulary. A simple example will illustrate it's basic usage:

```
<rdf:RDF xmlns:rdf="http://www.w3.org/1999/02/22-rdf-syntax-ns#"
  xmlns:foaf="http://xmlns.com/foaf/0.1/">

  <foaf:Person>

    <foaf:name>Peter Parker</foaf:name>

    <foaf:mbox rdf:resource="mailto:peter.parker@dailybugle.com"/>

  </foaf:Person>

</rdf:RDF>
```

In other words, there is a person, with the name "Peter Parker", who has an email address of "peter.parker@dailybugle.com".

Publishing data containing plain text email addresses is just asking for trouble; to avoid this FOAF defines another property, `foaf:mbox_sha1sum` whose value is a SHA1 encoded email address complete with the `mailto:` URI scheme prefix. The FOAF project wiki has a handy reference page pointing to a number of [different ways of generating a SHA1 sum](#).

Other Properties

```
<foaf:Person>

  <foaf:name>Peter Parker</foaf:name>

  <foaf:gender>Male</foaf:gender>

  <foaf:title>Mr</foaf:title>

  <foaf:givenname>Peter</foaf:givenname>

  <foaf:family_name>Parker</foaf:family_name>

  <foaf:mbox_sha1sum>cf2f4bd069302febd8d7c26d803f63fa7f20bd82</foaf:mbox_sha1sum>

  <foaf:homepage rdf:resource="http://www.peterparker.com"/>

  <foaf:weblog rdf:resource="http://www.peterparker.com/blog/">

</foaf:Person>
```

Which is a slightly richer description of Peter Parker, including some granularity in the markup of his name through the use of `foaf:title`, `foaf:givenname`, and `foaf:family_name`. We also now know that Peter Parker is male (`foaf:gender`) and has both a homepage (`foaf:homepage`) and a weblog (`foaf:weblog`).

4. Ontology and Reasoning

- The second way for encoding meaning is by the use of logical constructs.
- The standards enabling reasoning on the Semantic Web are RDF Schema, the ontology language OWL (Web Ontology Language) and RIF (Rule Interchange Format).
- The use of reasoning, that is, drawing conclusions from existing facts in the Semantic Web, a selection of ontology constructs are explained based on example, namely, `rdfs:subClassOf`, `owl:sameAs`, and property chains.
- The `rdfs:subClassOf` construct can be used to model class hierarchies. For example, the Music Ontology which specifies two classes, `mo:MusicGroup` and `mo:MusicArtist`, and contains an axiom specifying that `mo:MusicGroup` is a subclass of `mo:MusicArtist` via the `rdfs:subClassOf` property. Such a construct allows a reasoner to deduce that instances of `mo:MusicGroup` are also of type `mo:MusicArtist`.
- OWL enables also more complex modeling constructs, for example, cardinality restrictions (a predicate can only have a certain number of objects) or disjoint classes (the set of instances of two classes are disjoint). Specialized software systems, so-called reasoners, are able to take OWL knowledge bases and (1) check for the consistency of the knowledge base and (2) infer new statements based on existing ones.